

Логика работы новых методов в MC tbcv-vip-prime-spo-aff

Оглавление

• [Логика работы новых методов в MC tbcv-vip-prime-spo-aff](#)

• [Новые АПИ для MC tbcv-vip-prime-spo-aff](#)

- [POST /vtbo/customers/by_mdm_id - СДЕЛАНО](#)
- [POST /vtbo/customer/by_mdm_id - СДЕЛАНО](#)
- [GET /vtbo/inner_circle/member- Сделано](#)
- [GET /vtbo/affiliate/family/check- Сделано](#)
- [GET /vtbo/master/family/check - Сделано](#)
- [POST /vtbo/inner_circle/check/by_customer - Сделано](#)
- [POST /vtbo/customer/servicepack - Сделано](#)
- [GET /vtbo/count_approved_invitation/check - пока не делаем ждем решения Ани нужна ли такая проверка](#)
- [GET /vtbo/inner_circle/check/{affiliate_id}](#)
- [POST /vtbo/count_invitation/check - пока не берем, ждем Лену - кол-во семейных на ui?](#)
- [GET /vtbo/aff_counter/list- СДЕЛАНО](#)
- [POST /vtbo/affiliate_invitation/add - готово все кроме метод ИС 2035_2 internal_bff \(в проработке\) + согласовать изменение события в ОС Нотификация](#)
- [POST /vtbo/affiliate_invitation](#)
- [POST /vtbo/prime/affiliate/approve - все готово кроме внешнего метода ИС 2035_2 \(в проработке\) + возможно не понадобится отправка пуша в Нотификацию](#)
- [DELETE /vtbo/prime/affiliate](#)

Новые АПИ для MC tbcv-vip-prime-spo-aff

Номер версии	YAML файл с контрактами	Комментарий
1	file-20260402-162603366.json	Добавлены методы: 1. GET /vtbo/customer/preferences 2. POST /vtbo/customer/by_mdm_id 3. POST /vtbo/customer/servicepack 4. GET /vtbo/aff_counter/list 5. DELETE /vtbo/prime/affiliate

2	file- 20260402- 162603367. json	Удален метод: POST /vtbo/customer/by_mdm_id Добавлены методы: 1. GET /vtbo/customers/by_mdm_id 2. GET /vtbo/inner_circle/member 3. GET /vtbo/master/family/check 4. GET /vtbo/affiliate/family/check 5. GET /vtbo/count_approved_invitation/check 6. GET /vtbo/inner_circle/check/{affiliate_id} 7. POST /vtbo/count_invitation/check 8. POST /vtbo/affiliate_invitation
3	file- 20260402- 162603367. json	1. Переименованы методы: a. GET /vtbo/master/family/check на GET /vtbo/affiliate/family/check b. GET /vtbo/affiliate/family/check на GET /vtbo/master/family/check 2. Удален метод GET /vtbo/customer/preferences - из ответа удален параметр preferenceTotalShare, метод перенесен в MC share 3. Изменены методы: a. DELETE /vtbo/prime/affiliate - изменена логика b. POST /vtbo/count_invitation/check - изменена логика c. GET /vtbo/inner_circle/member изменена логика d. GET /vtbo/master/family/check - изменена логика e. GET /vtbo/affiliate/family/check - изменена логика f. POST /vtbo/customer/servicepack - изменена логика 4. Добавлен метод: a. POST /vtbo/inner_circle/check/by_customer 5. Во всех методах кроме GET /vtbo/aff_counter/list добавлен параметр X-MDM-ID 6. Изменения в DTO: a. ClientRequestParams - убрана обязательность поля mdmId

4	affUi v4.json	Изменены методы: 1. GET/vtbo/affiliate/family/check - в ответ добавлен customerId 2. GET/vtbo/master/family/check - в ответ добавлен customerId 3. POST vtbo/customers/by_mdm_id - изменен operationId Добавлены методы: 1. POST vtbo/customer/by_mdm_id
5	affUi v5.json	Изменены методы: 1. POST /vtbo/affiliate_invitation - в ответе метода изменено DTO на InvitationInfoResponseParams 2. DELETE /vtbo/prime/affiliate - изменились параметры запроса метода и логика

1. [POST /vtbo/customers/by_mdm_id](#) -получить список ФИО и id клиентов по списку mdmId
2. [POST /vtbo/customer/by_mdm_id](#) - получить ФИО и id клиента по mdmId
3. [GET /vtbo/inner_circle/member](#) - определить роль клиента в Близком круге
4. [GET /vtbo/affiliate/family/check](#) - проверить наличие аффилированного лица в Группе Близкие ключевой персоны
5. [GET /vtbo/master/family/check](#) - проверить наличие ключевой персоны в Группе Близкие аффилированного лица
6. [POST /vtbo/inner_circle/check/by_customer](#) - проверка является ли близкий АЛ/КП и если он АЛ, то является ли он АЛ для пользователя
7. [POST /vtbo/customer/servicepack](#) - получить Пакет услуг клиента по mdmId
8. [GET /vtbo/count_approved_invitation/check](#) - проверка количества принятых приглашений у ключевой персоны за последние 30 дней
9. [GET /vtbo/inner_circle/check/{affiliate_id}](#) - проверка аффилированного лица и ключевой персоны перед созданием приглашения в Близкий круг
10. [POST /vtbo/count_invitation/check](#) - проверить количество доступных приглашений в Премияльный Близкий круг и Группу Близкие
11. [GET /vtbo/aff_counter/list](#) - получить матрицу возможных для добавления аффилированных лиц
12. [POST /vtbo/affiliate_invitation/add](#) - создать приглашение в Премияльный Близкий круг
13. [POST /vtbo/affiliate_invitation](#) - для получения приглашаемым данных по приглашению в Близкий круг
14. [POST /vtbo/prime/affiliate/approve](#) - для принятия/отклонения приглашения в Близкий круг
15. [DELETE /vtbo/prime/affiliate](#) - предоставляет возможность клиенту КП удалить АЛ из Близкого круга

[POST /vtbo/customers/by_mdm_id](#) - СДЕЛАНО

Метод предоставляет возможность получить список ФИО и id клиентов по списку mdmId
 Логика работы метода:

1. Вызываем метод GET /vtbo/sofk/customers/by_id MC tbcv-vip-prime-sofk - в запросе метода в массиве mdmIds добавляем массив mdmId из запроса метода:
 - a. Пришел ответ с http code 200 - возвращаем http code 200 в ответе метода
 - b. Пришел ответ с http code 404 - возвращаем http code 404 в ответе метода

POST /vtbo/customer/by_mdm_id - СДЕЛАНО

Метод предоставляет возможность получить ФИО и id клиента по mdmId

Логика работы метода:

1. Вызываем метод GET /vtbo/sofk/customer/{mdm_id} MC tbcv-vip-prime-sofk - в запросе метода указываем mdmId из запроса метода:
 - a. Пришел ответ с http code 200 - возвращаем http code 200 в ответе метода
 - b. Пришел ответ с http code != 200 - возвращаем http code 500 в ответе метода

GET /vtbo/inner_circle/member - Сделано

Метод предоставляет возможность определить роль клиента в Близком круге.

Логика работы метода:

1. Получаем mdmId клиента из хедера authorization в параметре x-mdm-id
2. Вызываем метод POST /vtbo/inner_circle/member/{mdm_id} MC tbcv-vip-prime-sofk - в запросе метода отправляем mdm_id полученный на шаге 1:
 - a. Пришел ответ с http code 200 - возвращаем http code 200 в ответе метода и набор параметров
 - b. Пришел ответ с http code 422 - возвращаем http code 422 в ответе метода

GET /vtbo/affiliate/family/check - Сделано

Метод предоставляет возможность проверить наличие аффилированного лица в Группе Близкие ключевой персоны

Логика работы метода:

1. Получаем mdmId клиента из хедера authorization в параметре x-mdm-id
2. Получаем mdmId всех АЛ - вызываем метод GET /vtbo/sofk/inner_circle MC tbcv-vip-prime-sofk - в запросе метода в параметре masterMdmId отправляем mdmId полученный на шаге 1:
 - a. Получили http code = 200 - переходим к шагу 3
 - b. Получили http code = 404 - возвращаем ошибку 404. Конец алгоритма.
 - c. Получили http code = 422 - возвращаем http code = 422 в ответе метода. Конец алгоритма.
3. Проверяем роль клиента в Группе Близкие - вызываем метод GetPerson ИС 1442 - в запросе метода направляем mdmId клиента:
 - a. Получили http code = 200:
 - i. Проверяем есть ли в массиве contactParentRelationship записи, удовлетворяющие условиям: relatedPartyUid = mdmId из запроса метода И contactParentRelationship.role = "50" И (EndDate = null или EndDate > текущей даты):
 1. Запись найдена - значит клиент = **дочка в Группе Близкие**, запоминаем PartyUid (владельца) из этой записи:
 - a. Получаем всех дочек Группы Близкие по владельцу - вызываем метод GetPerson ИС 1442 - в запросе метода направляем mdmId=PartyUid клиента из шага 3.а.i.1
 - i. Получили http code = 200: забираем из массива contactRelationship все relatedPartyUid (дочки), удовлетворяющие условиям: partyUid = mdmId из запроса метода И contactRelationship.role = "50" И

- contactRelationship.relatedAttrib2 = "Подтвержденная" И (EndDate = null или EndDate > текущей даты), переходим к шагу 4
 - ii. Получили http code != 200 - возвращаем ошибку 500. Конец алгоритма.
 - 2. Запись не найдена - проверяем есть ли в массиве contactRelationship записи, удовлетворяющие условиям: partyUid = mdmId из запроса метода И contactRelationship.role = "50" И contactRelationship.relatedAttrib2 = "Подтвержденная" И (EndDate = null или EndDate > текущей даты):
 - a. Записи найдены - значит клиент = **Владелец в Группе Близкие**, запоминаем все relatedPartyUid (mdmId дочек) и переходим к шагу 4
 - b. Записи не найдены - значит клиент = **не участник Группы Близкие**, переходим к шагу 5
 - b. Получили http code != 200 - возвращаем ошибку 500. Конец алгоритма.
- 4. Сравниваем mdmId полученные на шаге 2 с mdmId соответствующие relatedPartyUid полученными на шаге 3
 - a. Отправляем в ответе метода http code 200 и массив mdmId аффилированных лиц полученный на шаге 2 с параметрами:
 - i. mdmId по которым есть пересечения указываем isFamily = true
 - ii. mdmId по которым нет пересечений указываем isFamily = false
- 5. Отправляем в ответе метода http code 200 и массив mdmId аффилированных лиц полученный на шаге 2 с меткой isFamily = false

GET /vtbo/master/family/check - Сделано

Метод предоставляет возможность получить ключевую персону и проверить является ли аффилированное лицо участником Группы Близкие.

Логика работы метода:

1. Получаем mdmId клиента из хедера authorization в параметре x-mdm-id
2. Получаем mdmId ключевой персоны - вызываем метод GET /vtbo/sofk/inner_circle MC tbcv-vip-prime-sofk - в запросе метода в параметре affiliateMdmId указываем mdmId полученный на шаге 1:
 - a. Получили http code = 200 - переходим к шагу 3
 - b. Получили http code = 404 - возвращаем ошибку 404. Конец алгоритма.
 - c. Пришел ответ с http code 422 - возвращаем http code 422 в ответе метода
3. Проверяем роль в Группе Близкие - вызываем метод GetPerson ИС 1442 - в запросе метода направляем mdmId клиента из хедера:
 - a. Получили http code = 200:
 - i. Проверяем есть ли в массиве contactParentRelationship записи, удовлетворяющие условиям: relatedPartyUid = mdmId из запроса метода И contactParentRelationship.role = "50" И (EndDate = null или EndDate > текущей даты):
 1. Запись найдена - значит клиент = **дочка в Группе Близкие**, переходим к шагу 4
 2. Запись не найдена - проверяем есть ли в массиве contactRelationship записи, удовлетворяющие условиям: partyUid = mdmId из запроса метода И contactRelationship.role = "50" И contactRelationship.relatedAttrib2 = "Подтвержденная" И (EndDate = null или EndDate > текущей даты):
 - a. Записи найдены - значит клиент = **Владелец в Группе Близкие**, переходим к шагу 4
 - b. Записи не найдены - значит клиент = **не участник Группы Близкие**, переходим к шагу 4
4. Отправляем в ответе метода http code 200 и набор данных. Конец алгоритма.
 - a. Если на шаге 3 определили что клиент = **Владелец в Группе Близкие/дочка в Группе Близкие**, то в ответе метода отправляем id ключевой персоны + optionToAdd = null

- b. Если на шаге 3 определили что клиент = **не участник Группы Близкие**, то в ответе метода отправляем optionToAdd = false - метка что нельзя создавать Группу Близкие, так как клиент не имеет ГБ, но является АЛ в Премииальном Близком круге

POST /vtbo/inner_circle/check/by_customer - Сделано

Метод предоставляет возможность проверить является ли близкий АЛ/КП и если он АЛ, то является ли он АЛ для пользователя

Логика работы метода:

1. Получаем mdmId клиента из хедера authorization в параметре x-mdm-id
2. Вызываем метод GET /vtbo/sofk/inner_circle/member/{mdm_id} для определения роли близкого в Близком круге - в запросе метода подставляем mdmId полученный в запросе метода:
 - a. Получили http code = 200:
 - i. isMaster = true, то значит близкий = КП и имеет свой Близкий круг - передаем в ответе метода http code 200 + isMaster = true. Конец алгоритма
 - ii. isMaster = false, то значит близкий = АЛ - вызываем метод POST /vtbo/sofk/affiliate/check для проверки входит ли АЛ в Близкий круг текущего пользователя - в запросе метода в параметре masterMdmId указываем mdmId полученный из хедера, в параметре affiliateMdmId указываем mdmId из запроса метода:
 1. Пришел ответ с http code 204 - возвращаем http code 200 в ответе метода + isMaster = false + isInnerCircle = true. Конец алгоритма
 2. Пришел ответ с http code 400 - возвращаем http code 200 в ответе метода + isMaster = false + isInnerCircle = false. Конец алгоритма
 3. Пришел ответ с http code 500 - возвращаем http code 500 в ответе метода. Конец алгоритма
 - b. Получили http code = 422 - возвращаем в ответе метода http code 422. Конец алгоритма
 - c. Пришел ответ с http code != 200/422 - возвращаем http code 500 в ответе метода. Конец алгоритма

POST /vtbo/customer/servicepack - Сделано

Метод предоставляет возможность получить Пакет услуг клиента по mdmId.

Логика работы метода:

1. Проверяем заполнен ли параметр mdmId в запросе метода:
 - a. mdmId заполнен - переходим к шагу 2
 - b. mdmId не заполнен - забираем mdmId из параметра x-mdm-id в заголовка метода, переходим к шагу 2
2. Вызываем метод ИС 1503 для получения ПУ клиента
 - a. Если метод вернул http code = 200 - то в ответе метода направляем http code 200. Конец алгоритма.
 - b. Если метод вернул http code != 200 - отправляем в ответе метода http code = 500. Конец алгоритма

GET /vtbo/count_approved_invitation/check - пока не делаем ждем решения Ани нужна ли такая проверка

Метод предназначен для проверки количества принятых приглашений у ключевой персоны за последние 30 дней

Логика работы метода:

1. Получаем mdmId ключевой персоны из хедера authorization в параметре x-mdm-id
2. Вызываем метод GET /vtbo/sofk/customer/{mdm_id} MC tbcv-vip-prime-sofk:
 - a. Пришел ответ с http code 200 - переходим к шагу 3

- b. Пришел ответ с http code 404 - возвращаем http code 404 в ответе метода. Конец алгоритма
- 3. Находим записи в таблице `affiliate_invitation_approved`, где `master_id` соответствует `customer_id` из ответа метода и `aproved = true` и `stamp` входит в диапазон: текущая дата - 30 дней:
 - a. Если количество записей $\geq$ 5 (добавить значение в *config*) - отдаем в ответе http code 200 и `isAllowed = "false"` и `declineReason = 'Нельзя добавлять или апгрейдить на Премиум условия более 5 клиентов'`. Конец алгоритма
 - b. Если количество записей $<$ 5 (добавить значение в *config*) - отдаем в ответе http code 200 и `isAllowed = "true"`. Конец алгоритма.

GET /vtbo/inner_circle/check/{affiliate_id}

Метод предназначен для проверки аффилированного лица и ключевой персоны перед созданием приглашения в Близкий круг

Логика работы метода:

1. Получаем `mdmId` ключевой персоны из хедера `authorization` в параметре `x-mdm-id`
2. Вызываем метод `GET /vtbo/sofk/customer/{mdm_id}` MC `tbcv-vip-prime-sofk`:
 - a. Пришел ответ с http code 200 - переходим к шагу 3
 - b. Пришел ответ с http code 404 - возвращаем http code 404 в ответе метода. Конец алгоритма
3. Проверяем, не отправляется ли повторно приглашение в Близкий круг - клиент переданный в запросе метода в параметре `affiliate_id` как `affiliate_invitation.affiliate_id` и `customerId` ключевой персоны полученный на шаге 2.а как `affiliate_invitation.master_id` и для этой записи `approved = null` и (`status != sent_family` или *или* `status != no_need_to_sent_family`):
 - a. Запись есть - отдаем в ответе http code 200 и `isAllowed = "false"` и `declineReason = 'Клиенту уже направлено приглашение в Близкий круг'`. Конец алгоритма
 - b. Записи нет - отдаем в ответе http code 200 и `isAllowed = "true"`. Переходим к шагу 4
4. Вызываем метод `POST /vtbo/sofk/inner_circle/check` MC `tbcv-vip-prime-sofk` - в запросе метода отправляем `affiliateId` из запроса метода и `master_id = customerId` ключевой персоны полученный на шаге 2.а:
 - a. Пришел ответ с http code 400 - отдаем в ответе http code 200 и `isAllowed = "false"` и `declineReason = message` из ответа метода. Конец алгоритма
 - b. Пришел ответ с http code 204 - отдаем в ответе http code 200 и `isAllowed = "true"`. Конец алгоритма

POST /vtbo/count_invitation/check - пока не берем, ждем Лену - кол-во семейных на ui?

Метод предоставляет возможность проверить количество доступных приглашений в Премияльный Близкий круг и Группу Близкие

Логика работы метода:

1. Получаем `mdmId` клиента из хедера `authorization` в параметре `x-mdm-id`
2. Проверяем какое значение параметра `familyCheck` пришло в запросе метода:
 - a. `familyCheck = true`, то вызываем метод `GetPerson` ИС 1442:
 - i. Получили http code = 200
 1. Находим в массиве `contactRelationship` записи, где:
 - a. `partyUid = mdmId` из запроса метода
 - b. И `contactRelationship.role = '50'`
 - c. И `contactRelationship.relatedAttrib2 = 'Подтвержденная'` ИЛИ `contactRelationship.relatedAttrib2 = 'Неподтвержденная'`
 - d. И (`EndDate = null` ИЛИ `EndDate >` текущей даты)
 2. Суммируем количество найденных записей

3. Рассчитываем кол-во возможных к добавлению участников по формуле: 5 - общее кол-во найденных записей. Записываем полученное значение в параметр `allowedFamilyCount` в ответе метода:
 - a. Если `allowedFamilyCount > 0` то переходим к шагу 3.
 - b. Если `allowedFamilyCount < или равно 0` то возвращаем в ответе метода `http code 400` и `message = 'Превышен лимит клиентов на добавление в Группу Близкие'`. Конец алгоритма
- ii. Получили `http code != 200` - возвращаем ошибку `500`. Конец алгоритма
- b. `familyCheck = false` - переходим к шагу 3
3. По `mdmId` из шага 1 проверяем количество аффилированных лиц (добавленных через ВТБО и ВТБ Про) - вызываем метод `GET /vtbo/sofk/count_affiliates/{mdm_id}` MC `tbcv-vip-prime-sofk`:
 - a. Получили `http code = 200`:
 - i. Если `totalAff` из ответа метода $>$ или равно 5 (*заложить в config*) - отдаем в ответе `http code = 200 + isAllowed = "false"` и `declineReason = 'Превышен лимит клиентов на добавление в Близкий круг'`. Конец алгоритма.
 - ii. Если `totalAff` из ответа метода < 5 - переходим к шагу 4.
 - b. Получили `http code 422` - считаем `totalAff` равно 0 и переходим к шагу 4.
 - c. Получили `http code != 200/422` - отправляем в ответе метода `http code 500`. Конец алгоритма.
4. Вызываем внешний метод 1447_16 для получения текущих остатков ключевой персоны - `/api/v1/counters/{mdmId}?counter=currentBalance,dvsCurrentBalance,investsProducts,partnersProducts` - в запросе метода указываем `mdmId` полученное на шаге 1 - описание здесь [Фин.показатели клиента - REST](#)
 - a. Получили `http code 200`:
 - i. Суммируем полученные значения в параметре `amount` из ответа метода.
 - ii. Находим в таблице `affiliates_counter_vtbo` значение `affiliates_count` соответствующее для рассчитанного выше остатка (шаг 4.a.i):
 1. Если `affiliates_count = 0` - отдаем в ответе метода `http code = 200 + isAllowed = "false"` и `declineReason = 'Превышен лимит клиентов на добавление в Близкий круг'`. Конец алгоритма
 2. Если `affiliates_count > 0` - сравниваем значение `affiliates_count` с `totalAff` полученным на шаге 3:
 - a. Если `affiliates_count > totalAff`, то:
 - i. Рассчитываем разницу между `affiliates_count` и `totalAff` - записываем полученное значение в параметр `allowedAffiliatesCount` в ответе метода.
 - ii. Отдаем в ответе `isAllowed = "true" + allowedAffiliatesCount + текущий остаток + allowedFamilyCount` (если получили на шаге 1.a). Конец алгоритма
 - b. Если `affiliates_count < или равно totalAff`, то отдаем в ответе `http code = 200 + isAllowed = "false"` и `declineReason = 'Превышен лимит клиентов на добавление в Близкий круг'`. Конец алгоритма
 - b. Получили `http code = 204` - отдаем в ответе `http code = 200 + isAllowed = "false"` и `declineReason = 'Превышен лимит клиентов на добавление в Близкий круг'`. Конец алгоритма
 - c. Получили `http code != 200` - отдаем в ответе `http code = 200 + isAllowed = "false"` и `declineReason = 'Не получили текущие остатки для расчета лимитов участников для добавление в Близкий круг'`. Конец алгоритма

GET /vtbo/aff_counter/list- СДЕЛАНО

Метод предоставляет возможность получить матрицу возможных для добавления аффилированных лиц из таблицы `affiliates_counter_vtbo` MC `tbcv-vip-prime-spo-aff`

**POST /vtbo/affiliate_invitation/add - готово все кроме метод ИС 2035_2 internal_bff (в проработке) +
согласовать изменение события в ОС Нотификация**

Метод предоставляет возможность создать приглашение в Премияльный Близкий круг и Группу Близкие
Логика работы метода:

1. Получаем mdmId клиента из хедера authorization в параметре x-mdm-id
2. Вызываем метод GET /vtbo/sofk/customer/{mdm_id} MC tbcv-vip-prime-sofk - в запросе указываем полученный mdmId из токена:
 - a. Пришел ответ с http code 200 - запоминаем полученные customer_id и ФИО клиента и переходим к шагу 3
 - b. Пришел ответ с http code 404 - возвращаем http code 404 в ответе метода. Конец алгоритма
3. Проверяем какие входные параметры получили в запросе метода:
 - a. Если в запросе метода заполнен параметр affiliateId, то переходим к шагу 4
 - b. Если в запросе метода не заполнен параметр affiliateId, то вызываем метод POST /vtbo/sofk/customer/add MC tbcv-vip-prime-sofk для добавления клиента в таблицу customer - в запросе метода указываем affiliateMdmId из запроса метода:
 - i. Если пришел http code = 200 - переходим к шагу 5
 - ii. Если пришел http code != 200 - отправляем в ответе метода http code 500. Конец алгоритма
4. Вызываем метод POST /vtbo/sofk/inner_circle/check MC tbcv-vip-prime-sofk - в запросе метода добавляем: в параметре affiliateId - полученный в запросе метода affiliate_id, в параметре masterId - полученный в ответе метода GET /vtbo/sofk/customer/{mdm_id} customer_id ключевой персоны (см. шаг 2.a):
 - a. Пришел ответ с http code 204 - переходим к шагу 5
 - b. Пришел ответ с http code 400 - возвращаем http code 400 и message из ответа метода
 - c. Пришел ответ с http code = 500 - возвращаем http code 500 в ответе метода
5. Проверяем какие входные параметры получили в запросе метода в параметре familyInvitation:
 - a. Если familyInvitation = true, то переходим к шагу 6
 - b. Если familyInvitation = false, то переходим к шагу 9
6. В рамках одной транзакции:
 - a. Добавляем связку аффилированного лица и ключевой персоны в таблицу affiliate_invitation, заполняем параметры:
 - i. external_id - формируется уникальный идентификатор на Бэке
 - ii. master_id - полученный в ответе метода GET /vtbo/sofk/customer/{mdm_id} customer_id ключевой персоны (см. шаг 2.a)
 - iii. affiliate_id - полученный в запросе метода affiliateId, если не получили affiliateId в запросе метода - то берем customerId из ответа метода POST /vtbo/sofk/customer/add (см. шаг 3.b.i)
 - iv. approved = null
 - v. create_dt = now()
 - vi. relative_name - полученный в запросе метода relativeName
 - vii. reason - пустой
 - viii. status = new
 - b. Формируем messages с event = 'TBCV_SOFK_push_affiliate_family' для отправки в кафку 1378 ОС Нотификация - описание здесь [Интеграция с 1378 ОС Нотификацияhttps://sfera.inno.local/knowledge/pages?id=1766429](https://sfera.inno.local/knowledge/pages?id=1766429):
 - i. Маппинг полей для добавления в message:
 1. client_id = affiliateMdmId из запроса метода
 2. parameters.clientName = Имя и первая буква фамилии ключевой персоны полученные на шаге 2.a
 3. parameters.deepLink = https://online.vtb.ru/i/affin?id=external_id, где external_id = affiliate_invitation.external_id

с. Добавляем запись в таблицу `send_to_notification_queue`, маппинг полей:

- i. `send_to_notification_queue.external_id` = `affiliate_invitation.external_id`
- ii. `send_to_notification_queue.external_notification_uuid` = `get_random_uuid()`
- iii. `send_to_notification_queue.mdm_id` = `mdmId` аффилированного лица - `affiliateMdmId` из запроса метода
- iv. `send_to_notification_queue.status` = всегда проставляем `NO_NEED_TO_SENT`
- v. `send_to_notification_queue.messages` = кладем сообщение сформированное на шаге 5.b

7. Коммитим изменения на шагах 5.a - 5.c

8. **Вызываем внешний метод ИС 2035_2 internal_bff (в проработке):**

а. Если пришел `http code` = 200, то

- i. Проставляем в таблице `affiliate_invitation` для записи `status` = `sent_family`
- ii. Проставляем в таблице `send_to_notification_queue` для записи `status` = `NEW`
- iii. Отдаем в ответе метода `http code` = 200 + диплинк из `send_to_notification_queue.messages.parameters.deepLink`. Конец алгоритма

б. Если пришел `http code` != 200 - проставляем в таблице `affiliate_invitation` для записи `status` = `sent_failed`, отдаем в ответе метода `http code` = 500. Конец алгоритма.

9. В рамках одной транзакции:

а. Добавляем связку аффилированного лица и ключевой персоны в таблицу `affiliate_invitation`, заполняем параметры:

- i. `external_id` - формируется уникальный идентификатор на Бэке
- ii. `master_id` - полученный в ответе метода `GET /vtbo/sofk/customer/{mdm_id}` `customer_id` ключевой персоны (см. шаг 2.а)
- iii. `affiliate_id` - полученный в запросе метода `affiliateId`, если не получили `affiliateId` в запросе метода - то берем `customerId` из ответа метода `POST /vtbo/sofk/customer/add` (см. шаг 3.b.i)
- iv. `approved` = null
- v. `create_dt` = `now()`
- vi. `relative_name` - полученный в запросе метода `relativeName`
- vii. `reason` - пустой
- viii. `status` = `no_need_to_sent_family`

б. Формируем `messages` с `event` = `'TBCV_SOFSK_push_affiliate_premium'` для отправки в кафку 1378 ОС Нотификация - описание здесь [Интеграция с 1378 ОС Нотификацияhttps://sfera.inno.local/knowledge/pages?id=1766429](https://sfera.inno.local/knowledge/pages?id=1766429):

i. Маппинг полей для добавления в `message`:

1. `client_id` = `affiliateMdmId` из запроса метода
2. `parameters.clientName` = Имя и первая буква фамилии ключевой персоны полученные на шаге 2.а
3. `parameters.deepLink` = `https://online.vtb.ru/i/affin?id=external_id`, где `external_id` = `affiliate_invitation.external_id`

с. Добавляем запись в таблицу `send_to_notification_queue`, маппинг полей:

- i. `send_to_notification_queue.external_id` = `affiliate_invitation.external_id`
- ii. `send_to_notification_queue.external_notification_uuid` = `get_random_uuid()`
- iii. `send_to_notification_queue.mdm_id` = `mdmId` аффилированного лица - `affiliateMdmId` из запроса метода
- iv. `send_to_notification_queue.status` = всегда проставляем `SENT`
- v. `send_to_notification_queue.messages` = кладем сообщение сформированное на шаге 5.b

д. Отправляем сообщение в в кафку 1378 ОС Нотификация:

i. В случае успешной отправки сообщения:

1. Коммитим изменения в таблице `affiliate_invitation` (шаг 5.а)
2. Отдаем в ответе метода `http code` = 200 + диплинк из `send_to_notification_queue.messages.parameters.deepLink`. Конец алгоритма

ii. В случае неуспешной отправки сообщения - отдаем в ответе метода `http code` = 500. Конец алгоритма.

POST /vtbo/affiliate_invitation

Метод предназначен для получения приглашаемым данных по приглашению в Близкий круг.

Логика работы метода:

1. Получаем mdmId клиента из хедера authorization в параметре x-mdm-id
2. Вызываем метод GET /vtbo/sofk/customer/{mdm_id} MC tbcv-vip-prime-sofk:
 - a. Пришел ответ с http code 200 - переходим к шагу 3
 - b. Пришел ответ с http code 404 - возвращаем http code 404 в ответе метода
3. Проверяем какой параметр заполнен в запросе метода:
 - a. Если пришел external_id - переходим к шагу 4
 - b. Если пришел masterMdmId, то вызываем метод GET /vtbo/sofk/customer/{mdm_id} MC tbcv-vip-prime-sofk - в запросе метода указываем mdm_id ключевой персоны из запроса метода:
 - i. Пришел ответ с http code 200 - переходим к шагу 4
 - ii. Пришел ответ с http code 404 - возвращаем http code 404 в ответе метода
4. Находим запись в таблице affiliate_invitation по external_id или (affiliate_id (customerId полученный на шаге 2.a) и master_id (customerId полученный на шаге 3.b.i)) и проверяем что для этой записи status = sent_family или status = no_need_to_sent_family и approved = null:
 - a. Запись найдена - переходим к шагу 5
 - b. Запись не найдена - отдаем в ответе http code = 400 и message = 'Приглашение недействительно'
5. Проверяем соответствует ли полученный на шаге 2.a customerId параметру affiliate_id найденной на шаге 4 записи в таблице affiliate_invitation
 - a. Нет - отдаем в ответе http code = 400 и message = 'Приглашение недействительно'
 - b. Да
 - i. Если в запросе метода external_id != null, то вызываем метод GET vtbo/sofk/customer/{customer_id} MC tbcv-vip-prime-sofk - в запросе метода в параметр customerId добавляем master_id ключевой персоны из таблицы affiliate_invitation:
 1. Пришел ответ с http code 200 - отдаем в ответе метода http code = 200 + данные ключевой персоны + externalId приглашения.
 2. Пришел ответ с http code 404 - возвращаем http code 404 в ответе метода
 - ii. Если в запросе метода external_id = null, то отдаем в ответе метода http code = 200 и данные ключевой персоны полученные на шаге 3.b.i + externalId приглашения.

POST /vtbo/prime/affiliate/approve - все готово кроме внешнего метода ИС 2035_2 (в проработке) + возможно не понадобится отправка пуша в Нотификацию

Метод предназначен для принятия/отклонения приглашения в Близкий круг

Логика работы метода:

1. Получаем mdmId клиента из хедера authorization в параметре x-mdm-id
2. Вызываем метод GET /vtbo/sofk/customer/{mdm_id} MC tbcv-vip-prime-sofk - в запросе указываем полученный mdmId из токена:
 - a. Пришел ответ с http code 200 - переходим к шагу 3
 - b. Пришел ответ с http code 404 - возвращаем http code 404 в ответе метода
3. Находим запись в таблице affiliate_invitation по externalId из запроса метода
4. Проверяем соответствует ли полученный на шаге 2.a customer_id параметру affiliate_id найденной на шаге 3 записи в таблице affiliate_invitation
 - a. Нет - отдаем в ответе http code = 400 и message = 'Приглашение недействительно'
 - b. Да - проверяем affiliate_invitation.approved = null и (affiliate_invitation.status = sent_family или affiliate_invitation.status = no_need_to_sent_family)?
 - i. Нет - отдаем в ответе http code = 400 и message = 'Приглашение недействительно'

ii. Да - переходим к шагу 5

5. Получаем mdmId ключевой персоны - вызываем метод POST /vtbo/customers/by_id MC tbcv-vip-prime-sofk - в запросе в массиве customerIds указываем master_id из записи полученной на шаге 3:
 - a. Пришел ответ с http code 200 - переходим к шагу 6
 - b. Пришел ответ с http code 404 - возвращаем http code 404 в ответе метода
6. Проверяем какое значение пришло в запросе метода в параметре decision:
 - a. true - переходим к шагу 7
 - b. false:
 - i. Проверяем какое значение параметра affiliate_invitation.status для записи найденной на шаге 3:
 1. Если affiliate_invitation.status = no_need_to_sent_family, то переходим к шагу 6.b.ii
 2. Если affiliate_invitation.status = sent_family, то **вызываем внешний метод ИС 2035_2 для отправки данных о принятии приглашения близким (в проработке):**
 - a. В **запросе метода отправляем:**
 - i. **mdmId ключевой персоны** полученный на шаге 5
 - ii. **mdmId аффилированного лица** полученный из токена
 - iii. **relativeName** - значение параметра relative_name записи полученной на шаге 3
 - b. **Пришел ответ с http code 200 - переходим к шагу 6.b.ii**
 - c. **Пришел ответ с http code != 200 - возвращаем http code 500 в ответе метода**
 - ii. В рамках одной транзакции:
 1. Проставляем для найденной записи approved = false и reason = 'Аффилированное лицо отклонило приглашение'
 2. **Ключевой персоне отправляем push-уведомление об отклонении приглашения в Близкий круг:**
 - a. **Формируем messages для отправки события с event = TBCV_SOFSK_affiliate_declined в кафку 1378 ОС Нотификация - описание здесь [Интеграция с 1378 ОС Нотификацияhttps://sfera.inno.local/knowledge/pages?id=1766429](https://sfera.inno.local/knowledge/pages?id=1766429):**
 - i. **parameters.clientName** = Имя и первая буква фамилии аффилированного лица полученные на шаге 2
 - ii. **client_id** - получаем mdmId ключевой персоны полученный на шаге 5
 - iii. **event** = 'TBCV_SOFSK_affiliate_declined'
 - b. **Добавляем запись в таблицу send_to_notification_queue, маппинг полей:**
 - i. **send_to_notification_queue.external_id** = affiliate_invitation.external_id
 - ii. **send_to_notification_queue.external_notification_uuid** = get_random_uuid()
 - iii. **send_to_notification_queue.mdm_id** = mdmId ключевой персоны полученный на шаге 5
 - iv. **send_to_notification_queue.status** = всегда проставляем NEW
 - v. **send_to_notification_queue.messages** = кладем сообщение сформированное на шаге 6.b.ii.2.a
 - iii. Коммитим изменения 6.b.ii. Отдаем в ответе метода http code = 204. Конец алгоритма
 7. Проверяем количество доступных приглашений в Близкий круг:
 - a. По mdmId ключевой персоны полученному на шаге 5 проверяем количество аффилированных лиц (добавленных через ВТБО и ВТБ Про) - вызываем метод GET /vtbo/sofk/count_affiliates/{mdm_id} MC tbcv-vip-prime-sofk:
 - i. Получили http code = 200:
 1. Если totalAff из ответа метода > 5 (*заложить в config*) - отдаем в ответе метода http code 400 и message = 'Превышен лимит клиентов на добавление в Близкий круг'. Конец алгоритма.
 2. Если totalAff из ответа метода < или равно 5 - переходим к шагу 7.b.
 - ii. Получили http code 422 - считаем totalAff равно 0 и переходим к шагу 7.b.

- iii. Получили http code != 200/422 - отправляем в ответе метода http code 500. Конец алгоритма.
- b. Вызываем внешний метод 1447_16 для получения текущих остатков ключевой персоны - /api/v1/counters/{mdmId}?counter=currentBalance,dvsCurrentBalance,investsProducts,partnersProducts - в запросе метода подставляем mdmId полученный на шаге 5 - описание здесь [Фин.показатели клиента - REST](#)
 - i. Получили http code 200:
 - 1. Суммируем полученные значения в параметре amount из ответа метода.
 - 2. Находим в таблице affiliates_counter_vtbo значение affiliates_count соответствующее для рассчитанного выше остатка (шаг 7.b.i.1):
 - a. Если affiliates_count = 0 - отдаем в ответе метода http code 400 и message = 'Превышен лимит клиентов на добавление в Близкий круг'. Конец алгоритма
 - b. Если affiliates_count > 0 - сравниваем значение affiliates_count с totalAff полученным на шаге 7a:
 - i. Если affiliates_count > totalAff, то переходим к шагу 8
 - ii. Если affiliates_count < или равно totalAff, то отдаем в ответе метода http code 400 и message = 'Превышен лимит клиентов на добавление в Близкий круг'. Конец алгоритма
 - ii. Получили http code = 204 - отдаем в ответе метода http code 400 и message = 'Превышен лимит клиентов на добавление в Близкий круг. Конец алгоритма
 - iii. Получили http code != 200 - отдаем в ответе метода http code 400 и message = 'Не получили текущие остатки для расчета лимитов участников для добавления в Близкий круг'. Конец алгоритма
- 8. Проверяем какое значение параметра affiliate_invitation.status для записи найденной на шаге 3:
 - a. Если affiliate_invitation.status = no_need_to_sent_family, то переходим к шагу 9
 - b. Если affiliate_invitation.status = sent_family, **вызываем внешний метод ИС 2035_2 для отправки данных о принятии приглашения близким (в проработке):**
 - i. **В запросе метода отправляем:**
 - 1. **mdmId ключевой персоны полученный на шаге 5**
 - 2. **mdmId аффилированного лица полученный из токена**
 - 3. **relativeName - значение параметра relative_name записи полученной на шаге 3**
 - ii. **Пришел ответ с http code 200 - переходим к шагу 9**
 - iii. **Пришел ответ с http code != 200 - возвращаем http code 500 в ответе метода**
- 9. Вызываем метод POST /vtbo/sofk/prime/affiliate/add MC tbcv-vip-prime-sofk - в запросе указываем affiliateId, masterId, relative_name из записи по приглашению найденной на шаге 3:
 - a. Пришел ответ с http code 204 - переходим к шагу 10
 - b. Пришел ответ с http code != 204 - возвращаем http code 400 и message "Не удалось добавить клиента на условия "Премиум", клиент успешно добавлен на условиях "Стандарт"
- 10. В рамках одной транзакции:
 - a. Для записи в таблице affiliate_invitation заполняем параметр: approved = true
 - b. **Ключевой персоне отправляем push-уведомление о принятии приглашения в Близкий круг:**
 - i. **Формируем messages для отправки события с event = TBCV_SOFK_affiliate_accepted в кафку 1378 ОС Нотификация - описание здесь [Интеграция с 1378 ОС Нотификацияhttps://sfera.inno.local/knowledge/pages?id=1766429](#):**
 - 1. **parameters.clientName = Имя и первая буква фамилии клиента полученные на шаге 2**
 - 2. **client_id = masterMdmId ключевой персоны полученный на шаге 5**
 - 3. **event = 'TBCV_SOFK_affiliate_accepted'**
 - ii. **Добавляем запись в таблицу send_to_notification_queue, маппинг полей:**
 - 1. **send_to_notification_queue.external_notification_uuid = get_random_uuid()**
 - 2. **send_to_notification_queue.external_id = affiliate_invitation.external_id**

3. `send_to_notification_queue.mdm_id` = mdmId ключевой персоны полученный на шаге 5
 4. `send_to_notification_queue.status` = всегда проставляем NEW
 5. `send_to_notification_queue.messages` = кладем сообщение сформированное на шаге 10.b.i
11. Коммитим все изменения на шаге 10 и удаляем запись из таблицы `affiliate_invitation`:
- a. В случае возникновения ошибки возвращаем http code 400 в ответе метода и message "Не удалось добавить клиента на условия "Премиум", клиент успешно добавлен на условиях "Стандарт"
 - b. Если запись успешно удалена - отдаем в ответе метода http code = 204. Конец алгоритма

DELETE /vtbo/prime/affiliate

Метод предоставляет возможность клиенту КП удалить АЛ из Близкого круга или АЛ удалиться из Близкого круга.

Логика работы метода:

1. Получаем mdmId из параметра `x-mdm-id` в заголовка метода, переходим к шагу 2
2. Вызываем DELETE /vtbo/sofk/prime/affiliate MC tbcv-vip-prime-sofk, параметры запроса метода формируются на основании параметров запроса текущего метода:
 - a. Если в запросе метода пришел `affiliateMdmId`, то в запросе метода DELETE /vtbo/sofk/prime/affiliate MC tbcv-vip-prime-sofk указываем как `masterMdmId` полученный mdmId на шаге 1, как `affiliateMdmId` полученный `affiliateMdmId` из запроса метода DELETE /vtbo/prime/affiliate
 - b. Если в запросе метода пришел `masterMdmId`, то в запросе метода DELETE /vtbo/sofk/prime/affiliate MC tbcv-vip-prime-sofk указываем как `masterMdmId` полученный `masterMdmId` из запроса метода DELETE /vtbo/prime/affiliate, как `affiliateMdmId` полученный полученный mdmId на шаге 1
3. Проверяем что пришло в ответе метода DELETE /vtbo/sofk/prime/affiliate MC tbcv-vip-prime-sofk:
 - a. Пришел ответ с http code 204 - возвращаем в ответе http code 204. Конец алгоритма
 - b. Пришел ответ с http code = 400 - возвращаем в ответе http code 400 и message из ответа метода. Конец алгоритма
 - c. Пришел ответ с http code = 500 - возвращаем в ответе http code 500. Конец алгоритма